

MACOS CLICKFIX LAUNCHAGENT PERSISTENCE

Campaign hunt for launchctl bootstrapping
the source-observed
com.apple.softwareupdated LaunchAgent.

HUNTING QUERY

CONFIDENCE: MEDIUM

SOURCE-CONFIRMED BEHAVIOR

retain facts, isolate interpretation

DeviceProcessEvents

direct telemetry boundary

HUNT · VALIDATE · TUNE

no automatic malicious verdict

REPORT DATE

10 August 2026

VERSION

v1.0

PRIMARY TELEMETRY

Microsoft Defender XDR ·
DeviceProcessEvents

ATT&CK

T1543.001

Evidence-led dossier. Query status remains an untested implementation
sketch until executed in the target environment.

INDEPENDENT RESEARCH FORMAT · CONTROLLED UI/UX MASTER



EXECUTIVE ASSESSMENT

Decision-grade summary for detection engineering and SOC operations.

ASSESSMENT

EXECUTIVE SUMMARY

Use the exact persistence action as a high-value campaign pivot.

Huntress observed the malware stripping attributes, ad-hoc signing a payload and registering `com.apple.softwareupdated.plist` through `launchctl bootstrap`. Defender process telemetry can observe the exact command, but the plist name is mutable and legitimate LaunchAgents are common.

OPERATIONAL VALUE

Precise retrospective scoping with explicit limits.

The query selects a source-observed process and plist relationship without a hash, domain, threshold or inferred payload result. It remains Hunting because it relies on a campaign artifact and has not been executed in the target tenant.

8.8**01**

OPERATIONAL VALUE

**Medium****02**

DETECTION CONFIDENCE

**Hunting****03**

OPERATIONAL READINESS



SOURCE-CONFIRMED FACTS

Observed behavior

- A ClickFix command downloaded and executed a Bash loader on macOS.
- Huntress observed `launchctl bootstrap` against `com.apple.softwareupdated.plist`.
- The registered payload used a user Library path and masqueraded as `SoftwareUpdate`.

ANALYTICAL ASSESSMENT

What the evidence supports

- The exact `launchctl/plist` combination is a selective campaign lead.
- Process telemetry cannot prove the plist contents or successful persistence.
- The plist and executable names can be changed by the actor.

DETECTION ASSUMPTIONS

What must be validated

- MDE process creation telemetry covers the target macOS fleet.
- `ProcessCommandLine` is complete for `launchctl` events.
- Authorized software does not commonly use this exact Apple-like plist name.

Confidence boundary

A match proves only that `launchctl` was invoked with the documented plist path. It does not prove that the payload was malicious, loaded successfully or contacted the reported infrastructure.

02

THREAT MECHANICS

Source-observed chain and explicit observability boundaries.

ATTACK FLOW

01

ClickFix execution

The user pastes a Terminal command from a fake CAPTCHA flow.

02

Profile host

A Bash loader collects CPU, hardware and user information.

03

Stage payload

Architecture-specific Mach-O content is copied into Apple-like cache paths.

04

Prepare execution

Attributes are stripped and the payload is ad-hoc signed.

05

Bootstrap persistence

launchctl registers com.apple.softwareupdated.plist for the GUI user.

Observable core

A launchctl process command line containing bootstrap and the user LaunchAgents path ending in com.apple.softwareupdated.plist.

Non-observable boundary

Plist content, BTM state, successful load, payload behavior, credential theft, wallet transfer and C2 response.

DETECTION HYPOTHESIS

The documented persistence action should create a selective process event.

If an attacker repeats the documented persistence step, DeviceProcessEvents should show launchctl bootstrapping com.apple.softwareupdated.plist from a user LaunchAgents path. Legitimate activity can be distinguished through software ownership, plist and payload collection, signer status and surrounding process/file/network evidence.

REQUIRED TELEMETRY

DeviceProcessEvents

- Microsoft Defender for Endpoint process events on macOS.
- FileName and ProcessCommandLine.
- Device and unique process/event identifiers.

OPTIONAL ENRICHMENT

Context that improves disposition

- Initiating process and account context.
- Plist and payload collection from the endpoint.
- File, signing, BTM and network timeline evidence.

UNAVAILABLE HERE

Do not infer

- Plist body from the process event.
- Successful launchd registration or execution result.
- Credential theft or wallet-drain outcome.



TELEMETRY & EVIDENCE MODEL

Minimum data contract and quality gates for a defensible hunt.

DATA CONTRACT

PRIMARY DATA SOURCE

Microsoft Defender XDR · DeviceProcessEvents

DeviceProcessEvents records process creation with image, path, command line, account, device, initiating-process and event identifiers.

EVIDENCE BOUNDARY

A matching event is a lead, not a verdict.

The table can establish invocation and command-line shape. It cannot establish plist contents, launchd success or malicious payload behavior.

REQUIRED AND ENRICHMENT FIELD MODEL

CATEGORY	FIELD	PURPOSE	REQUIREMENT
Process event	DeviceProcessEvents	Primary launchctl execution evidence.	REQUIRED
Executable	FileName, FolderPath	launchctl identity and path.	REQUIRED
Command	ProcessCommandLine	bootstrap and source-observed plist path.	REQUIRED
Device	DeviceId, DeviceName	Endpoint pivot.	REQUIRED
Process identity	ProcessId, ProcessUniqueId	Timeline pivot.	REQUIRED
Parent	InitiatingProcessFileName, InitiatingProcessCommandLine	Origin context when populated.	OPTIONAL
Event identity	Timestamp, ReportId	Documented event correlation.	REQUIRED

Data quality requirements

- Confirm current MDE coverage on supported macOS versions.
- Run a controlled benign launchctl bootstrap test.
- Verify full command-line retention.
- Collect the matching plist before disposition.

Known failure conditions

- MDE process telemetry is absent on the endpoint.
- Command line is truncated or redacted.
- The actor changes the plist name or persistence method.
- Activity predates retention.



DETECTION LOGIC

Primary-platform logic with every condition traceable to evidence.

HUNTING QUERY

DETECTION OBJECTIVE

Find the source-observed LaunchAgent registration.

Select launchctl processes whose command line contains bootstrap and the exact com.apple.softwareupdated.plist path.

ANALYTICAL LOGIC

Direct filters and investigable output.

Three direct predicates; no IOC list, join, sequence, threshold, rarity, aggregation or automatic attribution.

KQL · UNTESTED HUNT

```
// Hunting query - Untested implementation sketch
// Primary platform: Microsoft Defender XDR advanced hunting
// Telemetry: Microsoft Defender for Endpoint DeviceProcessEvents on macOS
// Source-observed behavior: launchctl bootstraps the campaign LaunchAgent plist.
// A match is campaign-specific persistence evidence, not proof of payload execution.
DeviceProcessEvents
| where FileName =~ "launchctl"
| where ProcessCommandLine contains "bootstrap"
| where ProcessCommandLine contains "/Library/LaunchAgents/com.apple.softwareupdated.plist"
| project Timestamp,
    DeviceId,
    DeviceName,
    AccountName,
    AccountUpn,
    FileName,
    FolderPath,
    ProcessCommandLine,
    ProcessId,
    ProcessUniqueId,
    InitiatingProcessFileName,
    InitiatingProcessFolderPath,
    InitiatingProcessCommandLine,
    InitiatingProcessAccountName,
    InitiatingProcessUniqueId,
    ReportId
| order by Timestamp desc
```

REVIEW FIELDS · BASELINE LOCALLY · VALIDATE BEFORE PRODUCTION

STATUS	THRESHOLD	JOIN / SEQUENCE	PRIMARY KEY	DISPOSITION
Untested hunt	None	None	DeviceName + ProcessUniqueld + Timestamp	Manual triage

What a match means

An endpoint invoked the source-observed persistence command and requires immediate plist, payload and ownership validation.

What a match does not prove

Malicious plist content, successful persistence, credential theft, crypto theft, C2 activity or campaign attribution.



TRIAGE & INVESTIGATION

Evidence collection before escalation.

SOC WORKFLOW

NUMBERED WORKFLOW

- 01

Confirm the raw event.
Verify launchctl path, full command line, user, device and unique identifiers.
- 02

Collect the plist.
Inspect Label, ProgramArguments, KeepAlive, RunAtLoad, ownership and timestamps.
- 03

Collect the payload.
Validate path, hash, architecture, signer and extended attributes.
- 04

Rebuild the timeline.
Review Terminal, shell, curl, chmod, xattr, codesign and file events.
- 05

Inspect follow-on.
Search for payload execution, Keychain access and network activity.

Escalation gate

Escalate when the plist is unowned, references an unsigned or ad-hoc-signed Apple-like cache payload, and the surrounding timeline contains ClickFix-style shell, download or credential-access evidence.

Primary benign overlap

Software installers, enterprise management, developers and security tests legitimately use launchctl bootstrap. The exact plist name narrows overlap but requires ownership and file-content validation.

INVESTIGATION PIVOTS

Host
macOS version, owner and MDE health.

LaunchAgent
Plist content, timestamps and ownership.

Payload
Hash, signer, architecture and prevalence.

Parent
Terminal or shell lineage and command history.

Files
Cache-path creation and attribute changes.

Network
Payload or shell connections after bootstrap.

DISPOSITION CRITERIA

OUTCOME	MINIMUM EVIDENCE	ACTION
BENIGN	Approved, owned and repeatable activity.	Document and tune narrowly.
SUSPICIOUS	Unknown activity without corroborated malicious follow-on.	Deepen endpoint review.
LIKELY MALICIOUS	Unauthorized match plus coherent file, process, network or control-plane evidence.	Escalate incident response.



FALSE POSITIVES & VALIDATION

Operational controls required before promotion.

QUALITY GATE

FALSE-POSITIVE MATRIX

SCENARIO	WHY IT OVERLAPS	TUNING ACTION
Legitimate software installer	May register a user LaunchAgent with launchctl.	Require publisher, package receipt, owner and expected payload path.
Enterprise management	May bootstrap managed background tasks.	Validate MDM identity, change record and stable label.
Security testing	May reproduce the campaign command.	Scope to approved device, operator and test window.

Baseline plan

- Search the native retention window.
- Inspect every exact-name match.
- Classify software owners and payload paths.
- Measure command-line completeness.

Validation procedure

- Bootstrap a harmless test plist on an isolated Mac.
- Confirm DeviceProcessEvents records the exact path.
- Collect and compare the raw event fields.
- Review all historical matches with endpoint owners.

TEST CASES

CASE	EXPECTED RESULT
Controlled plist with the exact campaign name	Expected match with complete command line.
Different legitimate LaunchAgent name	No match.
launchctl bootout against the same label	No match because bootstrap is absent.
Command line missing from telemetry	Failure condition; reject implementation.

Hunting

Initial query

Candidate

Current status

Production

Measured precision

Optimized

Stable feedback

Success

The controlled event matches with complete fields and any historical matches are explainable or actionable.

Rejection

The exact path is not retained or legitimate software commonly uses the same label without a durable discriminator.

Failure

Controlled bootstrap produces no usable DeviceProcessEvents record.



ATT&CK, CONCLUSION & REFERENCES

Coverage statement, residual risk and document control.

CLOSURE

MITRE ATT&CK MAPPING

TECHNIQUE	NAME	COVERAGE STATEMENT
T1543.001	Create or Modify System Process: Launch Agent	Directly covers the source-observed use of launchctl to register a user LaunchAgent for persistence.

Detection coverage summary

Strong for the exact com.apple.softwareupdated.plist bootstrap command; weak for renamed plists, LaunchDaemons, login items or silent API-based registration.

Residual risk

The actor can change labels, paths and persistence mechanisms, while successful launch and downstream theft require additional endpoint and network evidence.

ANALYST CONCLUSION

A selective campaign hunt, not a production behavior rule.

Primary incident evidence and documented Defender fields support a Hunting query with Medium confidence. The exact artifact improves precision but limits durability; promotion requires controlled replay, raw-event verification and tenant-specific ownership tuning.

REFERENCES

- 1. Huntress, "Mac Malware Drains Crypto Wallets Via Fake CAPTCHA Scam," 6 August 2026; infection observed approximately March 2026 and retrospectively hunted in June 2026. <https://www.huntress.com/blog/mac-crypto-draining-malware>
- 2. Microsoft Learn, "DeviceProcessEvents table," accessed 10 August 2026. <https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-deviceprocessevents-table>
- 3. MITRE ATT&CK, T1543.001. <https://attack.mitre.org/techniques/T1543/001/>

DOCUMENT CONTROL

VERSION	DATE	STATUS	OWNER
v1.0	10 August 2026	Hunting	Detection Engineering

Publication control

Eight A4 pages. URLs are defanged. No customer data, credentials, HTML source or rendering assets are included in the public repository.